```
    "analysis": {
      "analyzer": {
        "stem_exclusion_english_analyzer":{
          "type":"english",
          "stem_exclusion":["authority","authorization"]
        }
      }
    }
  }
}
```

Once we've created the index with these settings, the next step is to test the indexing request. The following listing uses the `english` analyzer to test a piece of text.

Listing 7.23 Stem exclusion in action

```
POST index_with_stem_exclusion_english_analyzer/_analyze
{
  "text": "No one can challenge my authority without my authorization",
  "analyzer": "stem_exclusion_english_analyzer"
}
```

The tokens created by this code are our usual tokens plus two untouched words: "authority" and "authorization". These two words will be output along with the other "stemmed" words.

We can also further customize the language analyzers. We discuss customizing analyzers in the next section.

While most analyzers do what we want in most cases, sometimes we need to implement text analysis for additional requirements. For example, we may want to remove special characters like HTML tags from incoming text or avoid specific stop words. The job of removing HTML tags is taken care of by the `html_strip` character filter, but unfortunately, not all analyzers have it. In such cases, we can customize the analyzer by configuring the required functionality: we can add a new character filter like `html_strip` and perhaps enable the `stop` token filter. Let's see how to configure the `standard` analyzer to suit advanced requirements.

7.4 Custom analyzers

Elasticsearch provides a lot of flexibility when it comes to analyzers: if off-the-shelf analyzers don't cut it for us, we can create our own custom analyzers. They can mix and match existing components from a large stash in Elasticsearch's component library.

Usual practice is to define a custom analyzer in `settings` when creating an index with the required filters and a tokenizer. We can provide any number of character and token filters but only one tokenizer (see figure 7.12).

As the figure shows, we define a custom analyzer on an index by setting the type to `custom`. Our custom analyzer is developed with an array of character filters represented by the `char_filter` object and an array of token filters represented by the `filter` attribute.